

Workflows for any domain. **auto-sprint** is just the first.

A portable workflow engine for the fleet — any provider, any machine, hundreds of runs at a time. auto-sprint shows it running a software sprint end to end.

THE PROBLEM

The orchestration is locked inside one vendor's CLI

The sprint loop works. But it can only run **inside Claude Code** — while the fleet it should drive is already heterogeneous:

fleet-dev / fleet-rev claude ·
local

fleet-win11 / fleet-mac15
ssh

spark ssh ·
gpu/arm

opencode-toy self-hosted

pm-planner / doer / harvester agy + claude
flavors

WHAT WE HAVE

Any OS, local or SSH-remote, six LLM providers behind one **execute_prompt** interface.

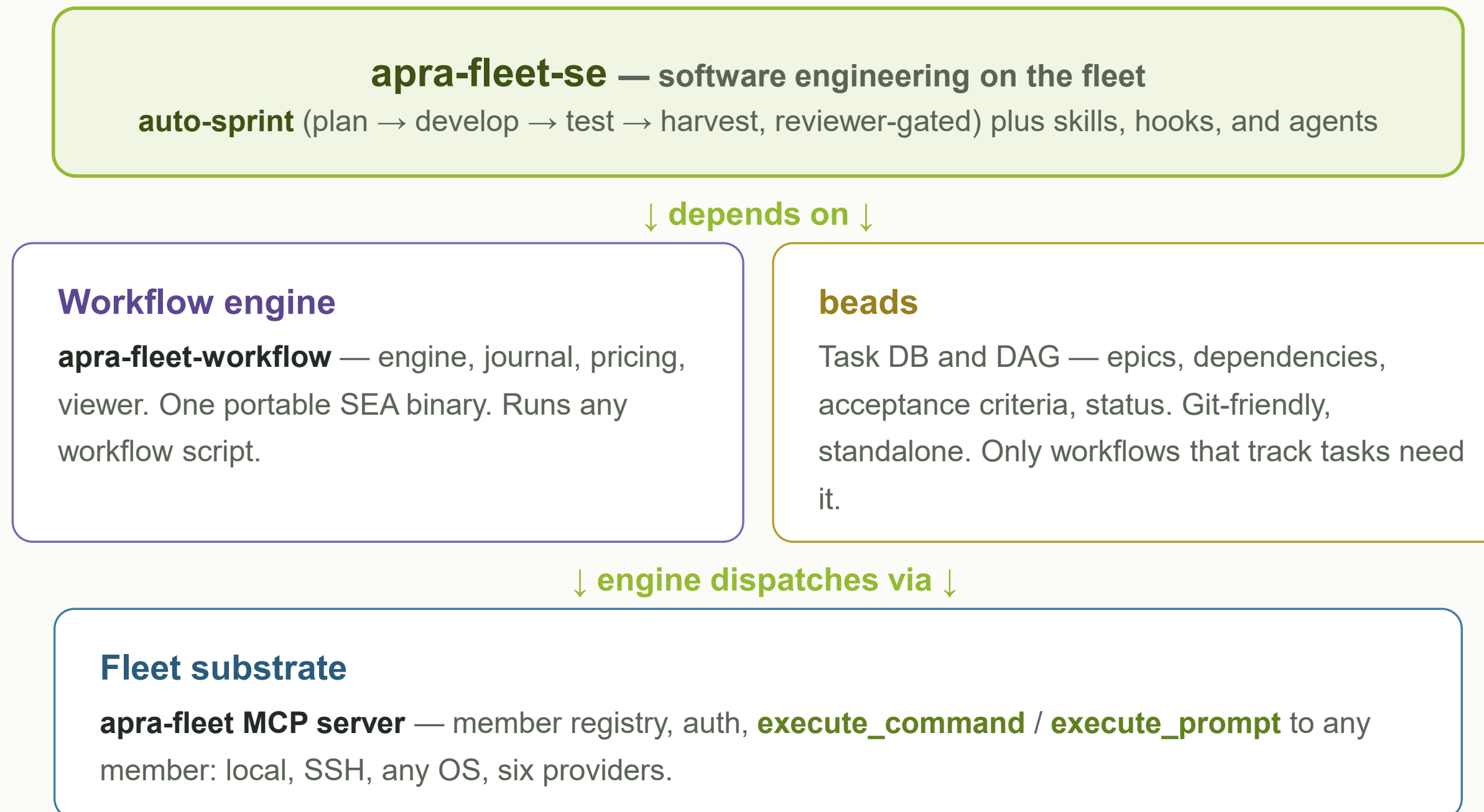
WHAT BLOCKS SCALE

A Claude-Code-shaped workflow only drives Claude-Code-shaped members. Concurrency ceiling = open CLI windows.

The fix: a portable workflow runtime that is not IDE-shaped.

Independent pieces — a workflow composes what it needs

Not a stack. The substrate, the task state, and the engine don't depend on each other. **apra-fleet-se** builds on all three — auto-sprint is part of it.



Observability (live dashboard, journals, cost stats) ships with the engine — every run gets it for free.

Three generations of the same sprint loop

The loop never changed. What changed is who drives it, and where it can run.

Gen 1 — /pm

Human-driven

You drive eight roles as subagents, cycle by cycle. Powerful, but manual — one provider, one project, full attention.

Gen 2 — /auto-sprint

Scripted, unattended

Retry/repair, phase resume, cost quotes — no human per cycle. But still inside Claude Code, still Claude-only members.

Gen 3 — auto-sprint

Portable, fleet-wide

Standalone CLI on a SEA binary. Any fleet member, embeds in CI, launchable by other agents via **execute_command**.

Gen 3 removes the "how many CLI windows can I keep open" ceiling.

Same sprint, now a portable runtime

Everything Gen 2 proved, extracted into the **apra-fleet-workflow** engine — one SEA binary with Node 22 embedded. **apra-fleet-se** builds on it for software engineering.

One binary, zero installs

Workflows are real files under `~/.apra-fleet/workflows/`. Ops boxes and CI runners need no Node, no npm.

Any provider

Dispatch via fleet's `execute_prompt` — claude, agy, codex, copilot, gemini, opencode. Tiers are `cheap/standard/premium`, not model IDs.

Any workflow

A `workflow.json` + entry script is the whole contract — **hello-world** is ~30 lines. Drop a dir in, it runs.

Agents launch agents

The CLI is addressable via fleet's own MCP tools — an orchestrator agent, cron job, or CI can start a run. No terminal, no human.

Landing as `apra-fleet workflow <name>` — SEA import spike green on Linux, macOS, and Windows.

The new engine is building its own next release

The all-provider auto-sprint is driving **apra-fleet-7pm** — a 14-task epic that builds the SEA workflow-runner subsystem.

```
node packages/apra-fleet-se/bin/cli.mjs -i apra-fleet-7pm -m pm-doer-std -b feat/workflow-runner -B main --goal P1/P2 --budget 50 --viewer-port 8090
```

Activity tree

Every dispatch and command, grouped by cycle and phase, down to raw input/output.

Beads panel

The live task DAG beside the run — auditable state, not chat scrollback.

LIVE via SSE

Watch a sprint from a browser tab while it runs unattended.

Cost + tokens

Per-activity token counts, run-level spend, and a `--budget` ceiling enforced by the runner.

Guardrails carried from Gen 2: topology preflight, branch-ensure, budget and cycle ceilings.

One engine, any domain

Nothing in the engine knows about code. auto-sprint is simply the first workflow — the software-development playbook. Any repeatable, agent-driven playbook fits the same contract.

Software development

auto-sprint — shipped and running today.

Retail

Catalog enrichment, pricing reviews, inventory reconciliation.

Banking

KYC checks, reconciliation runs, compliance reporting.

Healthcare

Claims triage, records summarization, coding review.

Customer support

Ticket triage, escalation playbooks, QA sampling.

...anything

If a playbook can be written down, the fleet can run it.

The authoring contract stays the same: `workflow.json` + entry script, dropped into `~/.apra-fleet/workflows/`.

WHY THIS WINS



From one AI pair-programmer to an agentic workforce

Single-IDE agent tools structurally cannot do this — their orchestration dies with the chat window. Ours is a runtime with durable state.

Portable

Any provider, any OS, local or remote. Swap a member's backend; the workflow doesn't change.

Embeddable

One SEA binary. Runs from a terminal, a CI job, or another agent's tool call.

Scalable

Fleet-wide dispatch — the path to hundreds of concurrent workflows.

Observable

Live dashboards per run; DAG + journals as auditable state.

Durable

State survives restarts, machine changes, provider swaps. Kill a run; resume the sprint.

"one developer, one AI session" → **"hundreds of concurrent, auditable, multi-provider agentic workflows"**

GETTING STARTED



You already know the pieces

Describe what you want — the LLM builds the invocation.

PLAIN ENGLISH

"Use /auto-sprint on epics BD-1 and BD-2, branch feat/my-sprint, close when P1 and P2 are done"

↓ LLM builds this

```
/auto-sprint {"issues":["BD-1","BD-2"],"branch":"feat/my-sprint","goal":"P1/P2"}
```

OPTIONS

goal P1 · P1/P2 · P1/P2/P3

max_cycles default 5

base_branch default main

requirementsFile optional

INSTALL APRA-PM

```
git clone https://github.com/Apra-Labs/apra-pm
&& node apra-pm/install.mjs
```

github.com/Apra-Labs/apra-pm